</ Data Mining:
Practical Python
for Beginners

/>

} /> [

Instructor:
Fateme Khodabande

fatemever2001@gmail.com

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

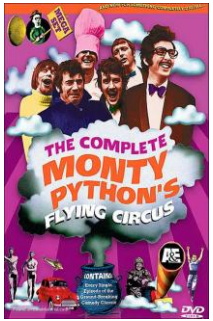
01

Introduction

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Background

- Python is a general-purpose programming language
- 1980s by Guido van Rossum at Centrum Wiskunde and Informatica (CWI) in the Netherlands
- The language is named after one of Guido's favorite programs "Monty Pythons Flying Circus"

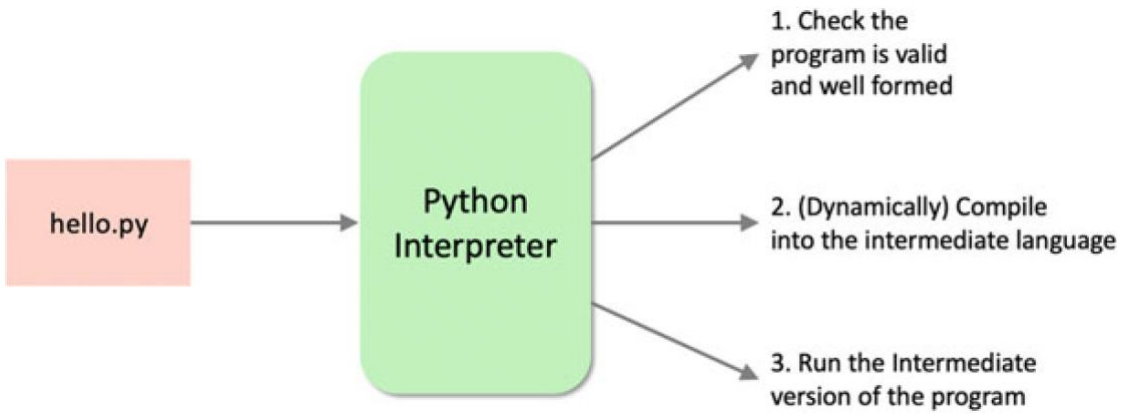


1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Increasing Interest in Python

1. Its flexibility and simplicity which makes it easy to learn.
2. Its use by the Data Science community where it provides a more standard programming language than some rivals such as R.
3. Its suitability as a scripting language for those working in the DevOps field where it provides a higher level of abstraction than alternative traditionally used languages.
4. Its ability to run on (almost) any operating system, but particularly the big three operating systems Windows, MacOS and Linux.
5. The availability of a wide range of libraries (modules) that can be used to extend the basic features of the language.
6. It is free!

Python Execution Model

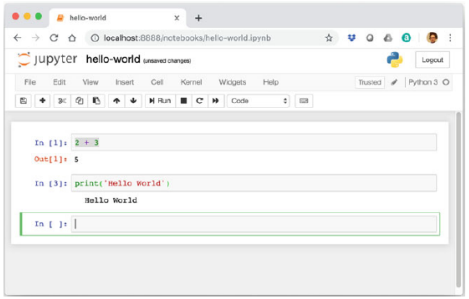


1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Running Python Programs

There are several ways in which you can run a Python program, including

- Interactively using the Python interpreter.
- Stored in a file and run using the Python command.
- Run as a script file specifying the Python interpreter to use within the script file.
- From within a Python IDE (Integrated Development Environment) such as PyCharm.
- Using Jupyter Notebooks in a web browser.



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1

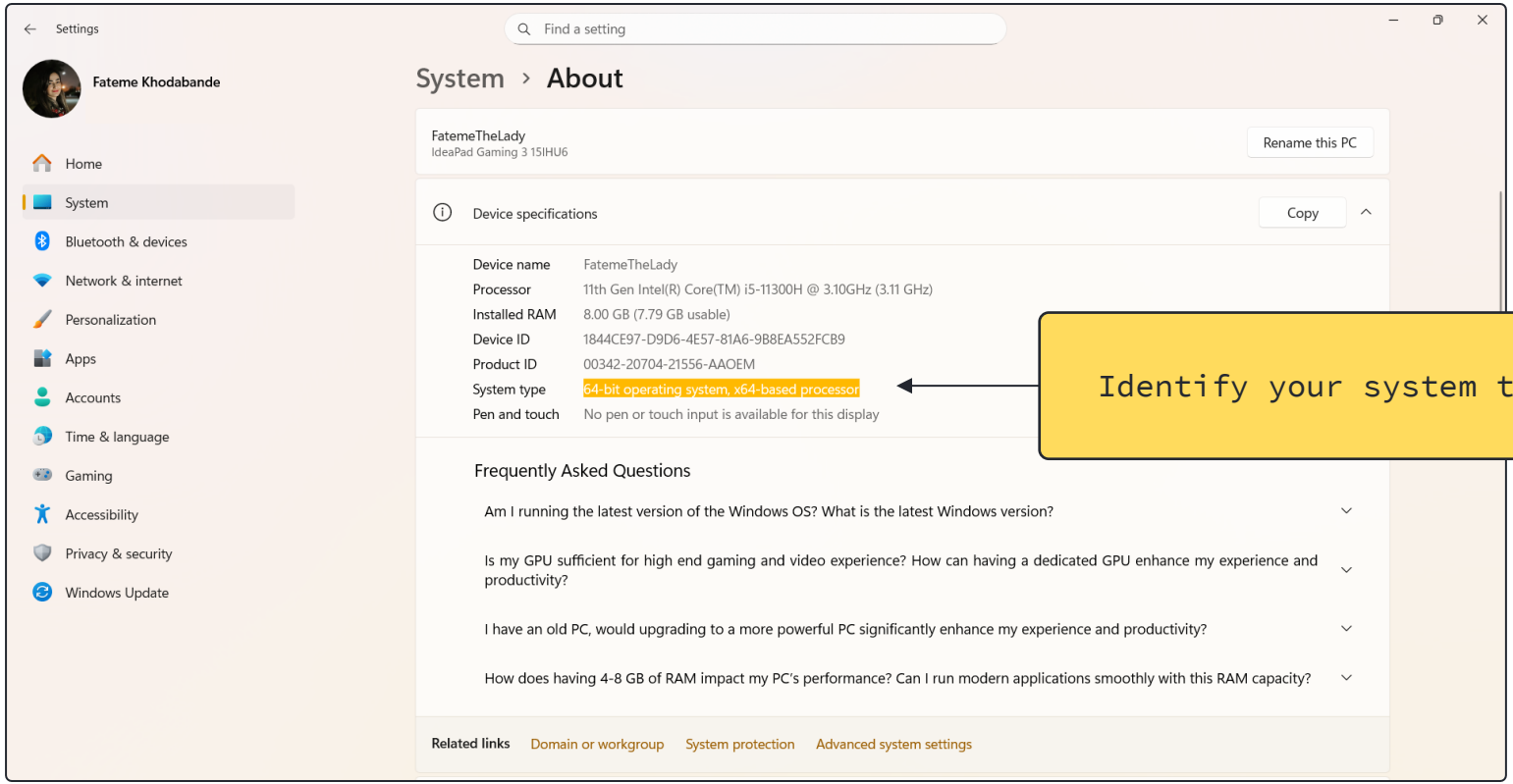
Where is Python Used

- Data Analytics
- Machine Learning and AI
- Database Work
- Animation
- Cross Platforms UI
- Academic Research
- Web Services
- …

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

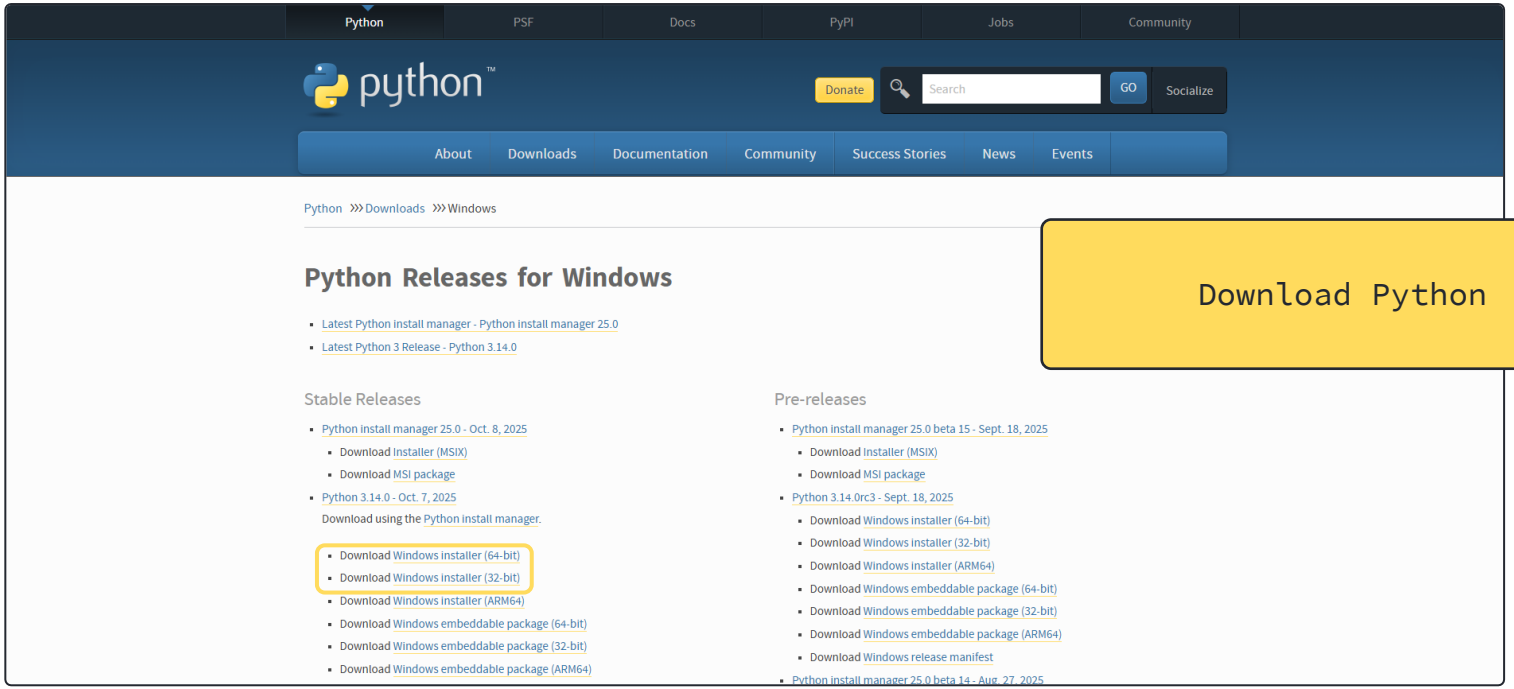
Setup Instructions

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

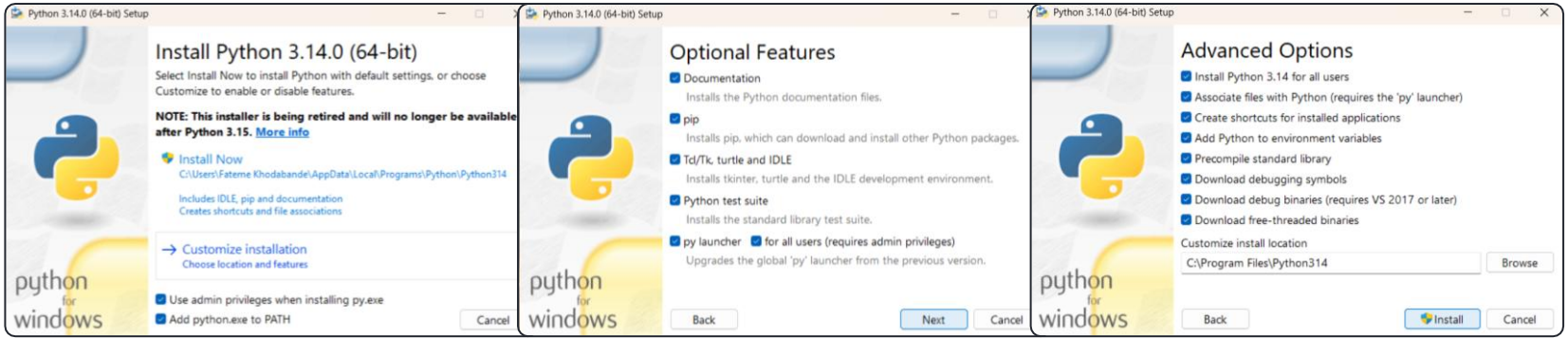
https://www.python.org/downloads/windows/



Download Python

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Install Python

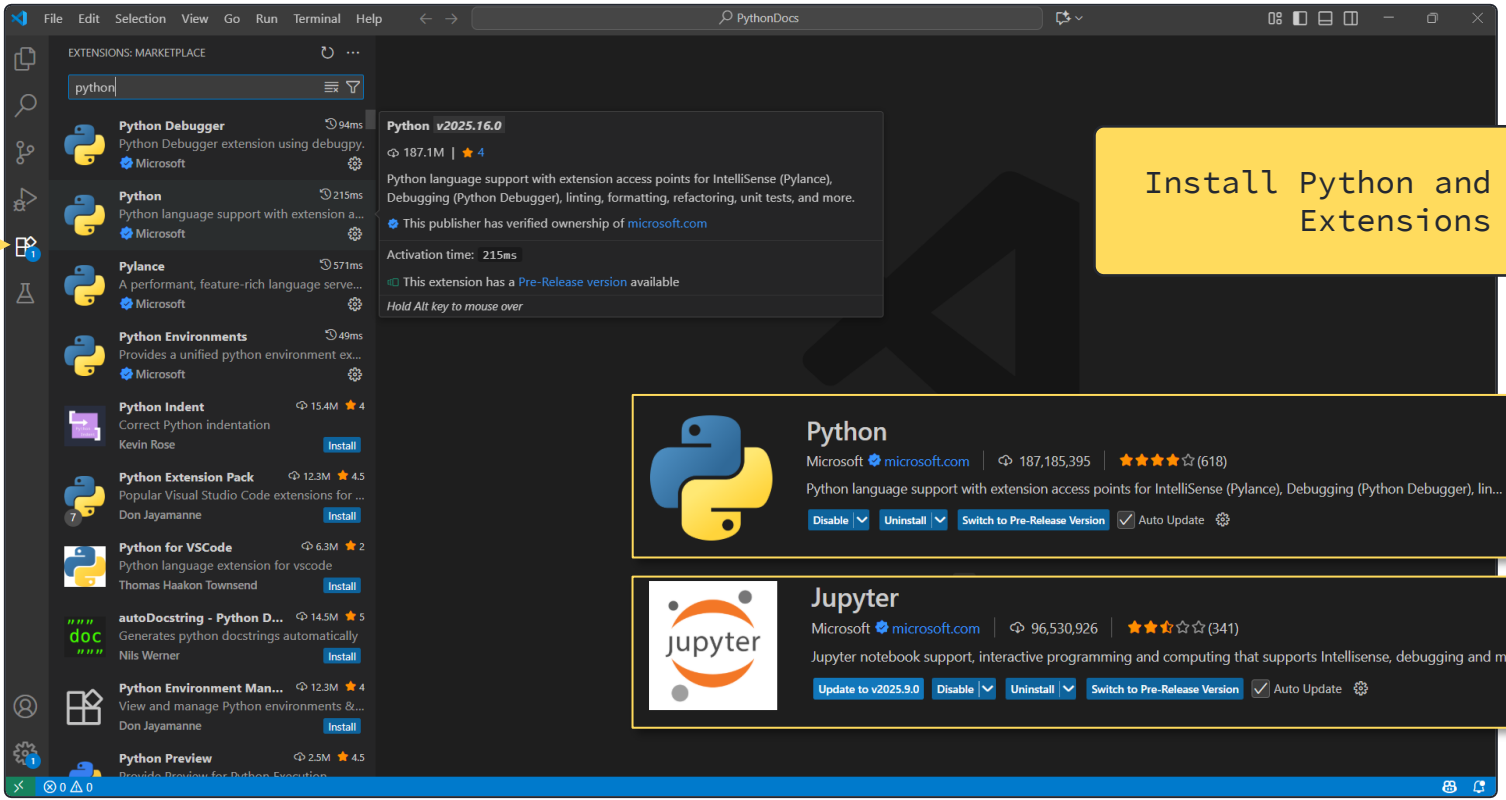


1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

https://code.visualstudio.com/Download

Download & Install VS Code

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

First Program

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

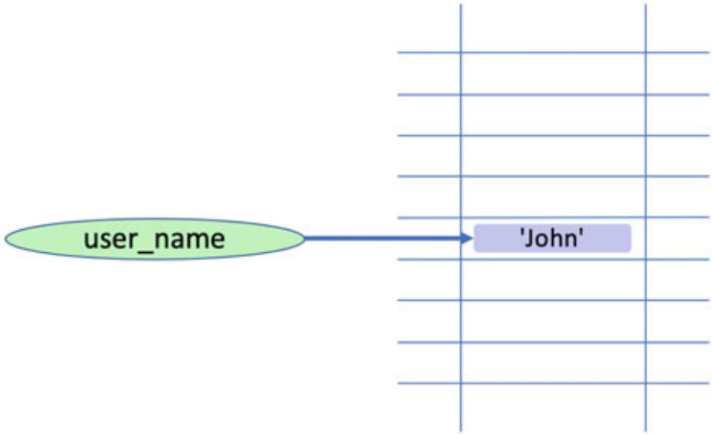
Reading User Input

- You can get the input from your user with `input()`.
- You can also tell the user what you want; For example if you are asking the user's name you can write: `input('Please Enter Your Name: ')`
- If you want to use the data that your user enters, you should save it in your memory, this can be done with what we call variables.
- The input() saves the user input as a string (We will talk about strings later) and you as a programmer should be careful about the type conversion yourself.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Variable

- A variable is a named area of the computers' memory that can be used to hold things (often referred to as data) such as strings, numbers, Boolean values such as True/False.
- It is called a variable because the value it references in memory can vary during the lifetime of the program.



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Dynamic Typing

- The type of the data held by a variable can dynamically change as the program executes. Although this may seem like the obvious way to do things, it is not the approach used by many programming languages such as Java and C# where variables are instead *statically* typed.
- It has pros and cons; But for many people the flexibility of Python's variables is one of its major advantages.

Examples

```
my_variable = 'John'
print(my_variable)
my_variable = 42
print(my_variable)
my_variable = True
print(my_variable)
```

The result of running the above example is

```
John
42
True
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Naming Convection

- Be all lowercase
- Be in general more descriptive than variable names such as a or b (there are some exceptions such as the use of variables *i* and *j* in looping constructs),
- With individual words separated by underscores as necessary to improve readability

Examples

- `my_name, your_name, user_name, account_name`
- `count, total_number_of_users, percentage_passed, pass_rate`
- `where_we_live, house_number,`
- `is_okay, is_correct, status_flag`

are all acceptable but

- `A, Aaaaa, aaAAAaa`
- `Myname, myName, MyName or MYName`
- `WHEREWELIVE`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

04

Data Types & Structures

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Python String

- In Python a string is a series, or sequence, of characters in order.
- A character is anything you can type on the keyboard in one keystroke, such as a letter ‘a’, ‘b’, ‘c’ or a number ‘1’, ‘2’, ‘3’ or a special character such as ‘\’, ‘[‘, ‘$’, and a space is also a character ‘ ‘, although it does not have a visible representation.
- It should also be noted that strings are immutable.

  Immutable means that once a string has been created it cannot be changed. If you try to change a string you will in fact create a new string, containing whatever modifications you made, you will not affect the original string in anyway.

- String Representation
  - ‘Hello World’
  - “Hello World”
  - "It’s the day"
  - ’She said "hello" to everyone’
  - """
    Hello
    World
    """

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

What Can You Do with Strings?

- String Concatenation
- Length of a String
- Accessing a Character
- Accessing a Subset of Characters
- Repeating Strings
- Splitting Strings
- Counting Strings
- Replacing Strings
- Converting Other Types into Strings
- Remove Prefix and Suffix
- Comparing Strings
- Other String Operations
- String Formatting Using f-string

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Numbers, Booleans

- Integers
  - e.g. 5
  - Converting input to integer
- Floating Point Numbers
  - e.g. 1.32
  - The largest floating point number that you can represent using the default built-in floating point type is 1.7976931348623157e+308
- Complex Numbers
  - e.g. 2j
- Boolean
  - A Boolean type can only be one of True or False (and nothing else). Note that these values are True (with a capital T) and False (with a capital F); true and false in Python are not the same thing and have no meaning on their own.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Arithmetic Operators

Operator	Description	Example
+	Add the left and right values together	`1 + 2`
–	Subtract the right value from the left value	`3 – 2`
*	Multiple the left and right values	`3 * 4`
/	Divide the left value by the right value	`12/3`
//	Integer division (ignore any remainder)	`12//3`
%	Modulus (aka the remainder operator)—only return any remainder	`13% 3`
**	Exponent (or power of) operator—with the left value raised to the power of the right	`3 ** 4`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Assignment Operators

Operator	Description	Example	Equivalent
+=	Add the value to the left-hand variable	x += 2	x = x + 2
-=	Subtract the value from the left-hand variable	x -= 2	x = x - 2
*=	Multiple the left-hand variable by the value	x *= 2	x = x * 2
/=	Divide the variable value by the right-hand value	x/= 2	x = x/2
//=	Use integer division to divide the variable's value by the right-hand value	x //= 2	x = x // 2
%=	Use the modulus (remainder) operator to apply the right-hand value to the variable	x %= 2	x = x % 2
**=	Apply the power of operator to raise the variable's value by the value supplied	x **= 3	x = x ** 3

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1

Exercise 1

- Ask the user about the distance he travelled in kilometer and convert it to mile and print it. (1km = 0.62 Mile)

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Tuples, Lists, Sets & Dictionaries

Tuples		Sets	
	Ordered		Unordered
	Immutable		Mutable
	Allow duplicates		Do not allow duplicates
	Indexed		Not indexed
	Fixed size		Growable
Lists		**Dictionaries**	
	Ordered		Ordered (from 3.7)
	Mutable		Mutable
	Allow duplicates		Allow duplicate values
	Indexed		Keyed
	Growable		Growable

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Tuples

- Tuples are an immutable ordered collection of objects; that is, each element in a tuple has a specific position (its index) and that position does not change over time. Indeed, it is not possible to add or remove elements from the tuple once it has been created.
- Tuples are defined using parentheses (i.e., round brackets '()') around the elements that make up the tuple, for example: tup1 = (1, 3, 5, 7)
- The tuple() function can also be used to create a new tuple from an *iterable*. An *iterable* is something that implements the *iterable* protocol. This means that a new tuple can be created from a *set*, a *list*, a *dict* (as these are all *iterable* types) or any type that implements the *iterable* protocol.
    - tuple(iterable)

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Tuples

- Accessing Elements of a Tuple
- Creating New Tuples from Existing Tuples
- Iterating Over Tuples
- Tuple Related Functions
- Checking if an Element Exists
- Nested Tuples
- Things You Can’t Do with Tuples

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Lists

- Creating Lists: Lists are created using square brackets positioned around the elements that make up the list. For example:
  - list1 = [’John’, ’Paul’, ’George’, ’Ringo’]
- The list() function can be used to construct a list from an iterable; this means that it can construct a list from a tuple, a dictionary or a set.
- If you want to construct a list from user *input,* you can use eval(); For example:
  - list_input = eval(input())

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Lists

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 0 1

Sets

- This collection is used to hold unique immutable collections of objects. It is a mutable collection and so the values it holds can change over time but does not allow duplicate values. It also supports basic set like operations such as intersection, union and difference.
- A set is defined using curly brackets (e.g., '{}'). For example,
  - basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
- As with tuples and lists Python provides a predefined function that can convert any *iterable* type into a set.
  - set(iterable)

Sets

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets

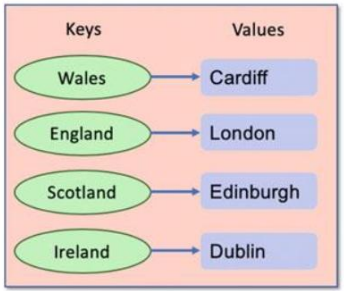
Method	Description
symmetric_difference_update()	Inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Dictionaries

- A dictionary (technically called a *dict* in Python) is a set of associations between a key and a value that is unordered, changeable (mutable) and indexed. This collection type allows for keyed access to values. That is, a value is stored in a dictionary using a key. This key is then used to retrieve that value at a later date. Dictionaries are very widely used in Python programs.
- A *dict* is created using curly brackets (‘{}’) where each entry in the dictionary is a key:value pair:

  cities = {’Wales’: ’Cardiff’, ’England’: ’London’, ’Scotland’: ’Edinburgh’, ’Northern Ireland’: ’Belfast’, ’Ireland’:’Dublin’}



- The dict() function can be used to create a new dictionary object froman iterable or a sequence of key:value pairs. The syntax of this function is: dict(**kwarg).

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1

Dictionaries

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and values
get()	Returns the value of the specified key
items()	Returns a list containing the tuple for each key-value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

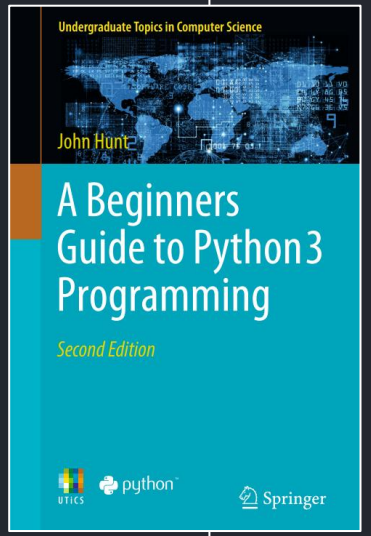
Exercise 2

Write a program that:

- Takes a list of numbers as input.
- Converts it into a set to remove duplicates.
- Stores the minimum, maximum, and average in a dictionary with keys "min", "max", and "avg".
- Prints the dictionary.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ Reference: A Beginners Guide to Python 3 Programming />



https://doi.org/10.1007/978-3-031-35122-8

1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1